SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: MLA0304
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO5: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr DUMPALA PRASANTH

QUESTIONS: 10 **Total Marks: 50**

Annexure A

Analytical Problem-Solving Assessment

Duration: 1hr

1. A warehouse robot must determine the shortest path to transport goods while avoiding obstacles. Apply **Dynamic Programming** to develop an optimal policy and explain how value iteration can improve navigation efficiency**.** **(5 Marks)**

2. A recommendation system collects customer interactions over multiple sessions. Apply **Monte Carlo Control** to update the policy and explain how the agent improves recommendations with experience. **.** **(5 Marks)**

3. A delivery robot operates in an uncertain environment. Apply the **SARSA** algorithm to update the action-value function and explain how the learned policy adapts to environmental changes. **.**
(5 Marks)

4. A robotic arm performs continuous object manipulation. Apply the **REINFORCE** algorithm to develop a policy that maximizes task completion accuracy. **.**

(5 Marks)

5. An autonomous vehicle is trained using **Q-Learning** and **Deep Q-Networks (DQN)**. Analyze the differences in learning performance, convergence speed, and scalability for complex driving environments. **.** **(5 Marks)**

6. A game-playing AI is implemented using **DQN**, **Double DQN (DDQN)**, and **Dueling DQN**. Analyze the advantages of DDQN and Dueling DQN over the standard DQN architecture. **. (5 Marks)**

7. A smart drone navigation system uses **A2C** and **A3C** algorithms. Analyze the impact of parallel learning on convergence speed, computational efficiency, and policy stability. **. (5 Marks)**

8. A healthcare decision-support system operates with incomplete patient information. Analyze how a Partially Observable Markov Decision Process (POMDP) improves decision-making compared to a standard MDP. **. (5 Marks)**

9. A robotic navigation system is trained using PPO, TRPO, and DDPG. Evaluate the suitability of each algorithm for continuous control tasks based on stability, sample efficiency, and computational complexity. **. (5 Marks)**

10. A smart city traffic management system uses **Multi-Agent Reinforcement Learning** (MARL) to coordinate traffic signals. Evaluate the **effectiveness of MARL** in improving traffic flow while considering scalability, coordination, fairness, and ethical challenges. **.**
(5 Marks)

Code of Conduct:

*I (*K Varshan / 192325089*) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

K Varshan
Signature of the student:

ANSWERS:

1. Dynamic Programming and Value Iteration for Warehouse Robot Navigation

Concept

The warehouse robot must find the shortest path to transport goods while avoiding obstacles. Dynamic Programming can solve this problem by dividing navigation into smaller state-based decisions. Value Iteration calculates the optimal value of each state and derives the best policy.

State

A state represents the robot's current position in the warehouse grid.

Actions

The robot can move:

- Up
- Down
- Left
- Right

Reward

- +10 for reaching the destination
- -10 for hitting an obstacle
- -1 for every movement step

Value Iteration

The value of each state is repeatedly updated using the Bellman optimality equation:

$V(s) = \max [R(s,a) + \gamma V(s')]$

The process continues until the values become stable. The action with the highest value becomes the optimal policy.

Application

Initially, the robot may not know which route is shortest. Value Iteration evaluates possible routes and assigns higher values to states that lead efficiently toward the destination.

Analysis

Value Iteration improves navigation because it considers future rewards rather than only the immediate reward. Obstacles receive negative rewards, encouraging the robot to select safe routes.

Conclusion

Dynamic Programming with Value Iteration can produce an optimal navigation policy that minimizes

unnecessary movements and avoids obstacles.

---

2. Monte Carlo Control for Customer Recommendation

Concept

Monte Carlo Control learns the value of actions from complete experiences or episodes. In a recommendation system, an episode can represent a sequence of customer interactions during one or more sessions.

State

The state may contain:

- Customer preferences
- Previous items viewed
- Previous purchases
- Current session information
- Interaction history

Actions

The actions are different products or content that can be recommended.

Reward

A positive reward can be given when the customer clicks, purchases, or positively interacts with a recommendation. A negative reward can be assigned when recommendations are ignored.

Application

After a complete customer session, the agent calculates the return obtained from each state-action pair and updates its estimated value.

For example:

Customer views product

↓

Recommendation shown

↓

Customer clicks

↓

Positive reward

↓

Update action value

↓

Improve future recommendation

Analysis

The advantage of Monte Carlo Control is that the agent learns directly from actual customer experiences. With more sessions, it can identify which recommendations produce better long-term engagement.

Conclusion

Monte Carlo Control can gradually improve recommendation quality by updating the policy using accumulated customer experience.

---

3. SARSA for Delivery Robot in an Uncertain Environment

Concept

SARSA is an on-policy reinforcement learning algorithm. It updates the action-value function using the current state, current action, reward, next state, and next action.

The update is:

$Q(s,a) = Q(s,a) + \alpha[R + \gamma Q(s',a') - Q(s,a)]$

State

The state can represent:

- Robot position
- Destination
- Nearby obstacles
- Battery level
- Environmental conditions

Actions

The robot can move:

- Up
- Down
- Left
- Right

Application

Suppose the robot takes an action and encounters an unexpected obstacle. SARSA uses the actual action selected in the next state to update its previous decision.

Current State

↓

Select Action

↓

Receive Reward

↓

Observe Next State

↓

Select Next Action

↓

Update Q-value

Analysis

Because SARSA is on-policy, it learns the value of the actions that the robot actually follows, including exploratory behavior. This can make the learned policy more cautious in uncertain environments.

Conclusion

SARSA is suitable for delivery robots because it can continuously update its policy as environmental conditions change.

---

4. REINFORCE for Robotic Arm Object Manipulation

Concept

REINFORCE is a policy-gradient algorithm that directly learns a policy. It adjusts the policy parameters so that actions leading to higher returns become more likely.

State

The state may include:

- Robotic arm position
- Object position
- Joint angles
- Gripper status
- Target position

Actions

The robotic arm can change its joint movements and gripper actions.

Reward

- Positive reward for correctly manipulating the object
- Negative reward for dropping the object
- Negative reward for incorrect movements
- Positive reward for completing the task accurately

Application

The robot performs several manipulation episodes. After an episode is completed, the total reward is calculated. The policy is then updated so that successful actions become more probable.

State

↓

Policy

↓

Action

↓

Object Manipulation

↓

Reward

↓

Policy Update

Analysis

REINFORCE is useful because it directly optimizes the policy rather than requiring a separate action-value table. However, its updates can have high variance, so many training episodes may be required.

Conclusion

REINFORCE can improve manipulation accuracy by increasing the probability of actions that contribute to successful object handling.

---

5. Q-Learning vs DQN for Autonomous Vehicle Training

Concept

Q-Learning and Deep Q-Networks both learn action values, but they differ in how those values are

represented.

Q-Learning

Q-Learning generally stores Q-values in a table:

State → Action → Q-value

It works well for small and discrete state spaces.

DQN

DQN uses a neural network to approximate Q-values. This makes it suitable for large state spaces such as complex driving environments.

Comparison

Factor	Q-Learning	DQN
State space	Small/discrete	Large/complex
Representation	Q-table	Neural network
Scalability	Limited	High
Memory	Can become large	More efficient for large spaces
Training	Simple	More computationally demanding
Complex driving	Less suitable	More suitable

Analysis

For a simple road grid, Q-Learning can converge quickly because the state space is small. However, an autonomous vehicle may need to process traffic, obstacles, road conditions, and sensor information. A Q-table becomes impractical in such situations.

DQN can generalize across large state spaces but requires more computational resources and careful training.

Conclusion

Q-Learning is suitable for simple environments, whereas DQN provides better scalability for complex autonomous driving environments.

---

6. DQN, Double DQN and Dueling DQN for Game AI

Concept

DQN estimates action values using a neural network. Double DQN and Dueling DQN improve different limitations of standard DQN.

Double DQN

Standard DQN can overestimate action values because the same values are used for selecting and evaluating actions.

Double DQN separates these roles:

- One network selects the action.
- Another network evaluates the selected action.

This reduces overestimation and can improve learning stability.

Dueling DQN

Dueling DQN separates the network into:

- State-value function
- Advantage function

It learns which states are valuable and which actions are advantageous within those states.

Comparison

Feature	DQN	Double DQN	Dueling DQN
Q-value estimation	Standard	Reduces overestimation	Separates value and advantage
Stability	Moderate	Improved	Improved
Action evaluation	Standard	More accurate	More informative
Complex games	Suitable	Better	Better

Analysis

Double DQN is particularly useful when overestimated Q-values affect learning. Dueling DQN is useful when many actions have similar effects but some states are more important than others.

Conclusion

DDQN improves value estimation, while Dueling DQN provides a better representation of state value and action advantage. Both can outperform standard DQN in suitable game environments.

---

7. A2C and A3C for Smart Drone Navigation

Concept

A2C and A3C are actor-critic algorithms. The actor selects actions, while the critic evaluates the quality of those actions.

A3C uses multiple parallel agents that interact with separate environments and update a shared model.

A2C

A2C uses synchronized parallel learning. Multiple workers collect experience and update the model together.

A3C

A3C uses asynchronous workers. Each worker interacts with its own environment and updates the shared network independently.

Impact of Parallel Learning

Convergence Speed:
Multiple environments generate experience simultaneously, allowing the agent to learn from more experiences in less wall-clock time.

Computational Efficiency:
Parallel workers can utilize multiple CPU cores and reduce the time required to collect training experience.

Policy Stability:
A2C can provide more synchronized and stable updates, while A3C benefits from diverse experiences collected by different workers.

Analysis

For drone navigation, parallel learning can expose the agent to different obstacle layouts and environmental conditions. This improves the diversity of training experiences.

Conclusion

A2C is attractive when synchronized and stable updates are desired, while A3C is useful when asynchronous parallel learning and diverse experience collection are beneficial.

---

8. POMDP for Healthcare Decision Support

Concept

A standard Markov Decision Process assumes that the current state is sufficiently observable. In healthcare, complete patient information may not always be available.

A Partially Observable Markov Decision Process (POMDP) handles uncertainty by maintaining a belief about the possible patient states.

MDP

Patient State → Action → Reward

POMDP

Hidden Patient State

↓

Observation

↓

Belief State

↓

Action

↓

Reward

State

The actual patient condition may include information that is not directly observable.

Observation

The system can observe:

- Symptoms
- Test results
- Vital signs
- Patient history

Application

If the patient's exact condition is uncertain, the system maintains probabilities over possible conditions and chooses an action based on this belief.

Analysis

POMDP provides better decision-making under incomplete information because it explicitly represents uncertainty. This is more realistic for healthcare than assuming all patient information is immediately available.

Ethical Consideration

The system should support healthcare professionals rather than independently making critical medical decisions. Patient privacy, fairness, transparency, and safety must also be considered.

Conclusion

POMDP is more suitable than a standard MDP when patient information is incomplete, uncertain, or noisy.

---

9. PPO, TRPO and DDPG for Continuous Robotic Control

Concept

The robotic navigation system requires continuous control. PPO, TRPO, and DDPG are RL algorithms that can be applied to control problems, with different approaches to stability and policy optimization.

PPO

PPO limits how much the policy can change during an update. This provides a good balance between stability, simplicity, and performance.

TRPO

TRPO explicitly constrains policy updates using a trust-region approach. This provides strong policy-update stability but increases computational complexity.

DDPG

DDPG is an actor-critic method designed for continuous action spaces. It directly produces continuous actions.

Comparison

Factor	PPO	TRPO	DDPG
Continuous control	Suitable	Suitable	Highly suitable
Stability	High	Very high	Moderate
Sample efficiency	Good	Good	Good
Computational complexity	Moderate	High	Moderate
Implementation	Relatively simple	More complex	Moderate
Best use	General continuous control	Stable policy updates	Continuous actions

Analysis

PPO is often a practical choice because it provides stable learning without the complexity of TRPO. TRPO provides strong theoretical control over policy updates but requires more computation. DDPG is specifically useful when the action space is continuous.

Decision

For robotic navigation, PPO is a strong general-purpose choice because of its balance between stability and computational complexity. DDPG is also suitable when direct continuous action control is important, while TRPO is useful when highly controlled policy updates are required.

---

10. MARL for Smart City Traffic Signal Coordination

Concept

Multi-Agent Reinforcement Learning allows multiple traffic signals to act as individual agents. Each intersection can observe local traffic conditions while coordinating with neighboring intersections.

State

Each traffic agent can observe:

- Number of vehicles
- Queue length
- Current signal phase
- Neighboring traffic conditions
- Emergency vehicle information
- Pedestrian conditions

Actions

The agent can:

- Extend green time
- Reduce green time
- Change signal phase

Reward

The reward can consider:

- Reduced waiting time
- Reduced queue length
- Increased traffic flow
- Safe pedestrian crossing
- Emergency vehicle priority

Scalability

As the number of intersections increases, the number of agents also increases. This makes coordination more difficult. Hierarchical or decentralized approaches can help control the complexity.

Coordination

Neighboring traffic signals should share relevant information so that one intersection does not optimize its traffic flow at the expense of another.

Fairness

The system should not continuously prioritize one road while causing excessive waiting on another.

Rewards should consider balanced service among different directions and road users.

Ethical Challenges

The system should prioritize:

- Pedestrian safety
- Emergency vehicles
- Fair traffic treatment
- Transparency
- Safe operation

Analysis

MARL can improve traffic flow because multiple intersections can adapt simultaneously rather than following fixed signal timings. However, scalability, coordination, fairness, and safety constraints must be addressed.

Conclusion

MARL is effective for smart-city traffic management when agents are properly coordinated and safety and fairness constraints are incorporated into the learning process.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)

Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively